

# Layoffs Dataset Cleaning Project

## 1) Project Overview

### 1.1 Project Objective

The objective of this project was to clean and standardize a layoff dataset using MySQL to make it suitable for exploratory data analysis and visualization.

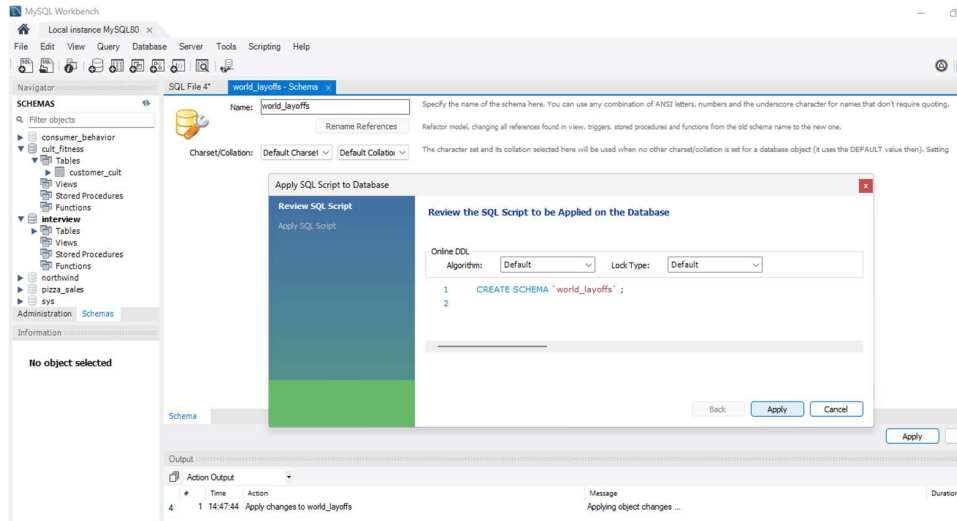
### 1.2 Dataset

Layoffs dataset collected from Kaggle containing company layoff records across industries and countries.

### 1.3 Tools Used

- MySQL Workbench
- SQL

## 2) Creating a Database



Created a schema named world\_layoffs

## 3) Importing CSV data to MySQL workbench

The dataset was imported into MySQL using the **Table Data Import Wizard** available in MySQL Workbench.

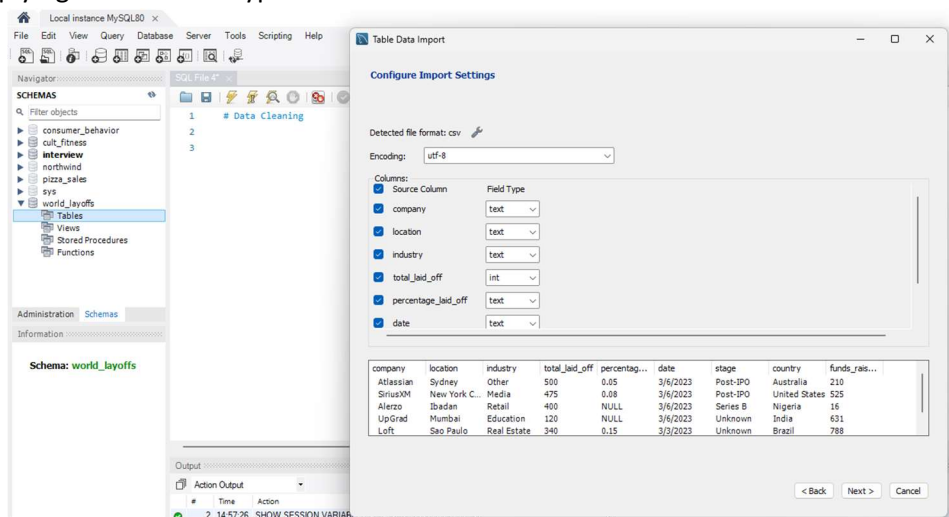
### Steps Performed

1. Right-clicked on **Tables** under the selected schema.
2. Chose **Table Data Import Wizard**.
3. Selected the CSV dataset file.
4. Configured import settings and reviewed column names.
5. Imported the dataset into a new table named layoffs.

### Important Note About Datatypes

During the import process, MySQL allows assigning datatypes to columns. However, some datatypes were intentionally not assigned correctly during import.

For example, the date column could have been assigned the DATE or DATETIME datatype, but it was intentionally kept as TEXT. This was done deliberately so that datatype conversion and standardization could be performed later during the data cleaning phase. This approach helps in identifying inconsistent or improperly formatted values before applying the final datatype transformations.



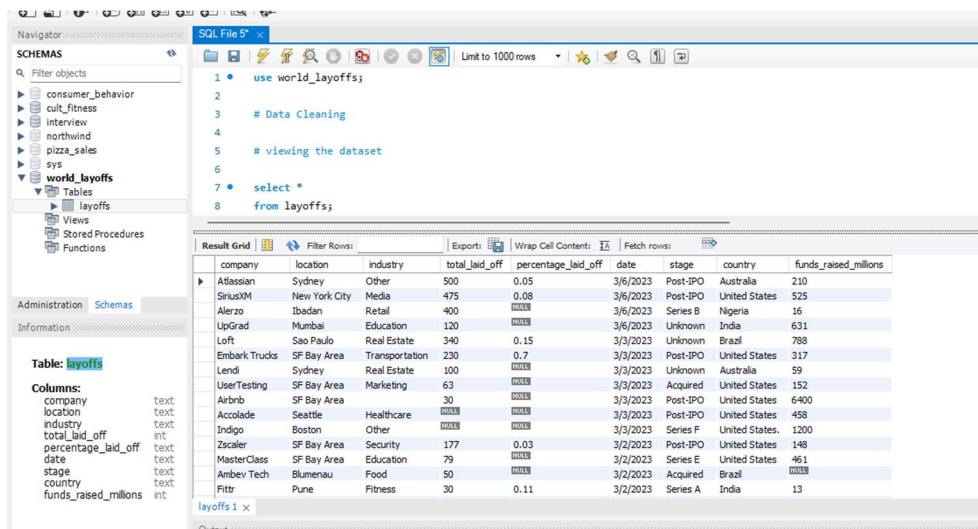
#### 4) Viewing the Imported Dataset

After importing the data, the table was refreshed and queried using,

*SELECT \* FROM layoffs;*

The dataset contained information such as:

- Company name – that did the layoffs
- Location - of the company
- Industry – that the company belong to or are a part of
- Total employees laid off
- Percentage laid off - employees in percentage that are laid off
- Date on which they laid off
- Company stage - how far along a company is in its growth journey.
- Country of the company located
- Funds raised - money a company has gotten from investors to grow.



## 5) Creating a Staging Table

Before performing any cleaning operations, a staging table named `layoffs_staging` was created.

This follows industry best practices because modifications should never be performed directly on raw data. The staging table acts as a working copy where all transformations and cleaning operations are applied safely.

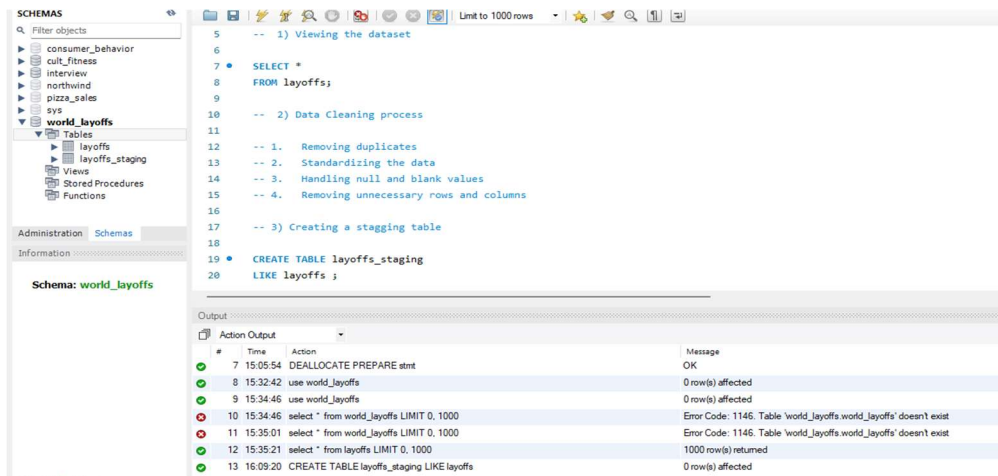
### SQL Used

```
CREATE TABLE layoffs_staging
LIKE layoffs;
```

```
INSERT INTO layoffs_staging
SELECT * FROM layoffs;
```

### Why This Was Done

- To Protect original raw data
- Allow rollback if mistakes occur
- Support safe experimentation during cleaning



## 6) Data Cleaning Process

The data cleaning process was divided into four major steps:

1. Removing duplicates
2. Standardizing data
3. Handling null and blank values
4. Removing unnecessary rows and columns

## 7) Identifying Duplicate Records

As there was no identifying factor in the dataset through which we can uniquely identify records to get the duplicate data so Duplicate rows were identified using the `ROW_NUMBER()` function with a Common Table Expression (CTE).

```

29  -- 4) Finding Duplicates
30
31  • SELECT * ,
32  ROW_NUMBER() OVER(PARTITION BY company,location,industry,total_laid_off,percentage_laid_off,'date',stage,country,funds_raised_millions) AS row_num
33  FROM layoffs_staging;
34
35  • WITH duplicate_cte AS
36  (
37  SELECT * ,
38  ROW_NUMBER() OVER(PARTITION BY company,location,industry,total_laid_off,percentage_laid_off,'date',stage,country,funds_raised_millions) AS row_num
39  FROM layoffs_staging
40  )
41  SELECT *
42  FROM duplicate_cte
43  WHERE row_num > 1;

```

| company          | location      | industry       | total_laid_off | percentage_laid_off | date       | stage    | country        | funds_raised_millions | row_num |
|------------------|---------------|----------------|----------------|---------------------|------------|----------|----------------|-----------------------|---------|
| Casper           | New York City | Retail         | 1000           | 100%                | 9/14/2021  | Post-IPO | United States  | 339                   | 2       |
| Cazoo            | London        | Transportation | 750            | 0.15                | 6/7/2022   | Post-IPO | United Kingdom | 2000                  | 2       |
| Hibob            | Tel Aviv      | HR             | 70             | 0.3                 | 3/30/2020  | Series A | Israel         | 45                    | 2       |
| Wildlife Studios | Sao Paulo     | Consumer       | 300            | 0.2                 | 11/28/2022 | Unknown  | Brazil         | 260                   | 2       |
| Yahoo            | SF Bay Area   | Consumer       | 1600           | 0.2                 | 2/9/2023   | Acquired | United States  | 6                     | 2       |

This query is used to identify duplicate records in the layoffs\_staging table.

The ROW\_NUMBER() window function assigns a sequential number (row\_num) to rows having the same values in columns such as:

- company
- industry
- location
- total\_laid\_off
- percentage\_laid\_off
- date
- stage
- country
- funds\_raised\_millions

Rows with identical values are grouped together using PARTITION BY.

In the second query, a Common Table Expression (CTE) named duplicate\_cte is created to store the result temporarily. The query then filters and displays only those rows where row\_num > 1, which indicates duplicate records.

## 8) Creating Secondary Staging Table

While removing duplicate records, I initially tried to delete duplicate rows directly from the Common Table Expression (CTE). However, MySQL generated an error because CTEs in MySQL are not updatable for DELETE operations.

The error occurred when attempting to run a query :

```

DELETE
FROM duplicate_cte
WHERE row_num > 1;

```

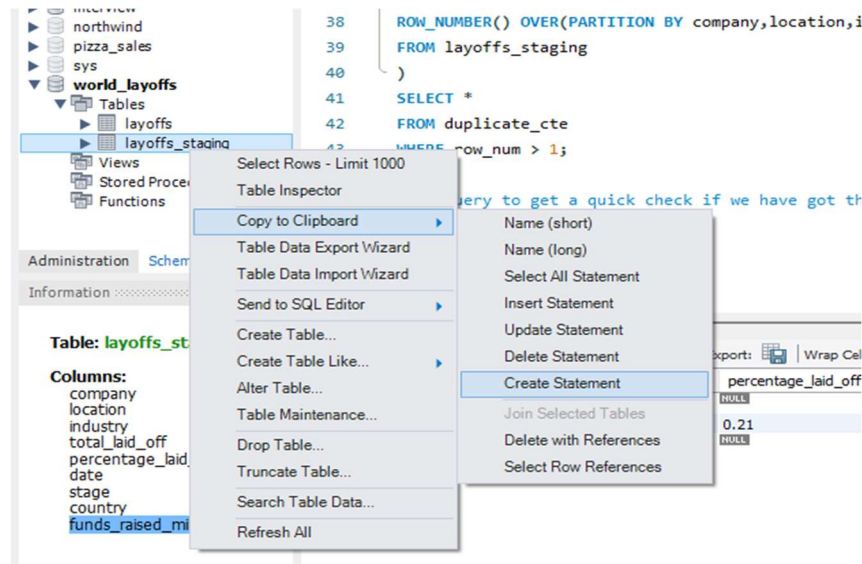
MySQL returned an error stating that the target table could not be updated.

To overcome this issue, another table named **layoffs\_staging2** was created. This new table included an additional column called row\_num, which stored the row numbers generated using the ROW\_NUMBER() function.

After inserting the data into this new table, duplicate rows were easily identified using the **row\_num** column, and rows where **row\_num > 1** were deleted successfully.

This approach helped safely remove duplicates while retaining one original copy of each record.

The structure of the existing layoffs\_staging table was copied using the **Create Statement** option in MySQL Workbench. An additional column named row\_num of datatype INT was added at the end of the table structure. This column was used to store row numbers for identifying duplicate records.



## SQL Used to Create the Table

```
CREATE TABLE `layoffs_staging2` (  
  `company` text,  
  `location` text,  
  `industry` text,  
  `total_laid_off` int DEFAULT NULL,  
  `percentage_laid_off` text,  
  `date` text,  
  `stage` text,  
  `country` text,  
  `funds_raised_millions` int DEFAULT NULL,  
  `row_num` int  
);
```

After creating the table, data from **layoffs\_staging** was inserted into **layoffs\_staging2** along with row numbers generated using the **ROW\_NUMBER()** function.

The **ROW\_NUMBER()** function assigned sequential numbers to rows having the same values across all important columns. Duplicate records received row numbers greater than 1.

```

66 • INSERT INTO layoffs_staging2
67   SELECT *,
68   ROW_NUMBER() OVER(
69     PARTITION BY company, location, industry, total_laid_off,
70     percentage_laid_off, `date`, stage, country,
71     funds_raised_millions
72   ) AS row_num
73   FROM layoffs_staging;
74
75 • SELECT *
76   FROM layoffs_staging2;

```

| company         | location      | industry       | total_laid_off | percentage_laid_off | date       | stage    | country       | funds_raised_millions | row_num |
|-----------------|---------------|----------------|----------------|---------------------|------------|----------|---------------|-----------------------|---------|
| E Inc.          | Toronto       | Transportation | NULL           | NULL                | 12/16/2022 | Post-IPO | Canada        | NULL                  | 1       |
| Included Health | SF Bay Area   | Healthcare     | NULL           | 0.06                | 7/25/2022  | Series E | United States | 272                   | 1       |
| 8Open           | Dublin        | Marketing      | 9              | 0.09                | 11/17/2022 | Series A | Ireland       | 35                    | 1       |
| #Paid           | Toronto       | Marketing      | 19             | 0.17                | 1/27/2023  | Series B | Canada        | 21                    | 1       |
| 100 Thieves     | Los Angeles   | Consumer       | 12             | NULL                | 7/13/2022  | Series C | United States | 120                   | 1       |
| 100 Thieves     | Los Angeles   | Retail         | NULL           | NULL                | 1/10/2023  | Series C | United States | 120                   | 1       |
| 10X Genomics    | SF Bay Area   | Healthcare     | 100            | 0.08                | 8/4/2022   | Post-IPO | United States | 242                   | 1       |
| Istdibs         | New York City | Retail         | 70             | 0.17                | 4/2/2020   | Series D | United States | 253                   | 1       |
| ZTM             | Sao Paulo     | Crypto         | 90             | 0.12                | 6/1/2022   | Unknown  | Brazil        | 250                   | 1       |
| ZTM             | Sao Paulo     | Crypto         | 100            | 0.15                | 9/1/2022   | Unknown  | Brazil        | 250                   | 1       |

## 9) Deleting Duplicate Rows

After identifying duplicate rows using the row\_num column, the duplicate records were removed from the **layoffs\_staging2** table.

```

79   -- 5) Removing duplicates
80
81 • DELETE
82   FROM layoffs_staging2
83   WHERE row_num > 1 ;
84
85 • SELECT *
86   FROM layoffs_staging2
87   WHERE row_num > 1 ;
88

```

| company | location | industry | total_laid_off | percentage_laid_off | date | stage | country | funds_raised_millions | row_num |
|---------|----------|----------|----------------|---------------------|------|-------|---------|-----------------------|---------|
|---------|----------|----------|----------------|---------------------|------|-------|---------|-----------------------|---------|

The duplicate rows were then deleted by removing all records where **row\_num > 1**. Since the first occurrence of each record had **row\_num = 1**, only the extra duplicate copies were removed.

Finally, a verification query was executed to confirm that no duplicate rows remained in the table. The query returned no records, confirming that all duplicate rows had been successfully removed.

## 10) Standardizing Data

Several inconsistencies in the dataset were corrected.

### 9.1 Removing Extra Spaces

Company names contained unnecessary spaces before or after the text.

These extra spaces could cause issues during filtering, grouping, and analysis.

Checking and removing extra spaces :



|     |                                |     |                                |
|-----|--------------------------------|-----|--------------------------------|
| 97  | -- removing extra spaces       | 96  | SELECT company , trim(company) |
| 98  |                                | 97  | FROM layoffs_staging2;         |
| 99  | SELECT company , trim(company) | 98  |                                |
| 100 | FROM layoffs_staging2;         | 99  | UPDATE layoffs_staging2        |
| 101 |                                | 100 | SET company = TRIM(company);   |
| 102 |                                | 101 |                                |
| 103 |                                | 102 | SELECT company                 |
|     |                                | 103 | FROM layoffs_staging2;         |

| company         | trim(company)   |
|-----------------|-----------------|
| E Inc.          | E Inc.          |
| Included Health | Included Health |
| &Open           | &Open           |
| #Paid           | #Paid           |
| 100 Thieves     | 100 Thieves     |
| 100 Thieves     | 100 Thieves     |
| 10X Genomics    | 10X Genomics    |

| company         |
|-----------------|
| E Inc.          |
| Included Health |
| &Open           |
| #Paid           |
| 100 Thieves     |
| 100 Thieves     |

Leading and trailing spaces were removed using the **TRIM()** function.

## 9.2 Standardizing Industry Names

During the data cleaning process, industry columns were checked for inconsistent values using the DISTINCT keyword. This helped identify different variations of the same industry name.

For example, values such as:

- Crypto
- Cryptocurrency
- Crypto Currency

represented the same industry but were written differently. If left unchanged, these values would be treated as separate industries during analysis and visualization, leading to inaccurate results.

**Before :**

```
107 • SELECT DISTINCT(industry)
108 FROM layoffs_staging2
109 ORDER BY industry ;
110
111 • SELECT *
112 FROM layoffs_staging2
```

| industry        |
|-----------------|
| NULL            |
| Aerospace       |
| Construction    |
| Consumer        |
| Crypto          |
| Crypto Currency |
| CryptoCurrency  |
| Data            |
| Education       |
| Energy          |
| Fin-Tech        |
| Finance         |
| Fitness         |
| Food            |

**After :**

```
111 • SELECT *
112 FROM layoffs_staging2
113 WHERE industry LIKE 'Crypto%';
114
115 • UPDATE layoffs_staging2
116 SET industry = 'Crypto'
117 WHERE industry LIKE 'Crypto%';
118
119 • SELECT DISTINCT(industry)
120 FROM layoffs_staging2
121 ORDER BY industry ;
```

| industry     |
|--------------|
| Aerospace    |
| Construction |
| Consumer     |
| Crypto       |
| Data         |
| Education    |
| Energy       |
| Fin-Tech     |
| Finance      |
| Fitness      |
| Food         |
| Hardware     |
| Healthcare   |
| HR           |

After the update, all cryptocurrency-related records were grouped under the single industry name Crypto, improving data consistency and accuracy for future analysis.

The location column was checked for inconsistencies, but no irregular values were identified.

## 9.3 Fixing Country Name Formatting

```

125 • SELECT DISTINCT country
126     FROM layoffs_staging2
127     ORDER BY country;

```

| country              |
|----------------------|
| South Africa         |
| South Korea          |
| Spain                |
| Sweden               |
| Switzerland          |
| Thailand             |
| Turkey               |
| United Arab Emirates |
| United Kingdom       |
| United States        |
| United States.       |
| Uruguay              |
| Vietnam              |

```

125 • SELECT DISTINCT country
126     FROM layoffs_staging2
127     ORDER BY country;
128
129 • UPDATE layoffs_staging2
130     SET country = TRIM(TRAILING '.' FROM country);
131
132 • SELECT DISTINCT country
133     FROM layoffs_staging2
134     ORDER BY country;

```

| country              |
|----------------------|
| United Arab Emirates |
| United Kingdom       |
| United States        |
| Uruguay              |
| Vietnam              |

Both values represent the same country, but due to the extra period (.), they appear as different records. Country names such as **United States.** were corrected by removing trailing periods.

## 10) Converting date column

- industry
- total\_laid\_off
- percentage\_laid\_off
- date
- stage

Administration Schemas

Information

Column: **date**

Definition: date date

Object Info Session

```

137 • SELECT *
138     FROM layoffs_staging2;
139
140
141 -- we can use str to date to convert it to date format
142 • UPDATE layoffs_staging2
143     SET `date` = STR_TO_DATE(`date`, '%m/%d/%Y');
144
145 -- now we can convert the data type properly
146 • ALTER TABLE layoffs_staging2
147     MODIFY COLUMN `date` DATE;
148

```

Output

Action Output

| #    | Time     | Action                                                                       |    |
|------|----------|------------------------------------------------------------------------------|----|
| ✓ 17 | 10:33:53 | UPDATE layoffs_staging2 SET country = TRIM(TRAILING '.' FROM country)        | 41 |
| ✓ 18 | 10:34:02 | SELECT DISTINCT country FROM layoffs_staging2 ORDER BY country LIMIT 0, 1000 | 59 |
| ✓ 19 | 11:56:21 | SELECT * FROM layoffs_staging2 LIMIT 0, 1000                                 | 10 |
| ✓ 20 | 11:57:12 | UPDATE layoffs_staging2 SET `date` = STR_TO_DATE(`date`, '%m/%d/%Y')         | 23 |
| ✓ 21 | 12:00:17 | ALTER TABLE layoffs_staging2 MODIFY COLUMN `date` DATE                       | 23 |

Initially, the date column was imported as TEXT. The values were first converted into proper date format using: **STR\_TO\_DATE( )** function. Then the column datatype was modified from TEXT to DATE.

### Why This Was Important ?

Proper date datatype is essential for : Time-series analysis, Sorting, Filtering by dates, Exploratory Data Analysis (EDA)

## 11) Handling Null and Blank Values



### 11.1 Identifying Null Values

Columns containing missing values were checked initially

- total\_laid\_off
- percentage\_laid\_off
- industry

```
2 • SELECT *
3 FROM layoffs_staging2
4 WHERE total_laid_off IS NULL;
5
6 • SELECT *
7 FROM layoffs_staging2
8 WHERE total_laid_off IS NULL
9 AND percentage_laid_off IS NULL;
```

| company      | location       | industry       | total_laid_off | percentage_laid_off | date       | stage       | country       | funds_raised_millions |
|--------------|----------------|----------------|----------------|---------------------|------------|-------------|---------------|-----------------------|
| Claravine    | Salt Lake City | Marketing      | NULL           | NULL                | 2020-03-31 | Seed        | United States | 4                     |
| Snap Finance | Salt Lake City | Finance        | NULL           | NULL                | 2020-03-31 | Unknown     | United States | NULL                  |
| Zerto        | Boston         | Infrastructure | NULL           | NULL                | 2020-03-31 | Series E    | United States | 130                   |
| Currency     | Los Angeles    | Finance        | NULL           | NULL                | 2020-04-01 | Unknown     | United States | NULL                  |
| GOAT Group   | Los Angeles    | Retail         | NULL           | NULL                | 2020-04-01 | Series D    | United States | 197                   |
| Levelset     | New Orleans    | Construction   | NULL           | NULL                | 2020-04-01 | Series C    | United States | 46                    |
| Capillary    | Bengaluru      | Retail         | NULL           | NULL                | 2020-04-02 | Private ... | India         | 102                   |
| Modsy        | SF Bay Area    | Retail         | NULL           | NULL                | 2020-04-02 | Series C    | United States | 70                    |
| Wonder       | New York City  | Other          | NULL           | NULL                | 2020-04-02 | Unknown     | United States | NULL                  |
| Alegion      | Austin         | Data           | NULL           | NULL                | 2020-04-03 | Series A    | United States | 16                    |
| AllyO        | SF Bay Area    | HR             | NULL           | NULL                | 2020-04-03 | Series B    | United States | 64                    |

It was observed that these rows contained very little useful information and would later be considered for deletion.

### 11.2 Checking Blank and Null Values in Industry column

It was noticed that the industry column contained both:

- blank values ('')
- null values (NULL)

```
161 -- looking for null and blank in industry column
162 • SELECT *
163 FROM layoffs_staging2
164 WHERE industry IS NULL
165 OR industry = '';
```

| company             | location    | industry | total_laid_off | percentage_laid_off | date       | stage    | country       |
|---------------------|-------------|----------|----------------|---------------------|------------|----------|---------------|
| Airbnb              | SF Bay Area |          | 30             | NULL                | 2023-03-03 | Post-IPO | United States |
| Bally's Interactive | Providence  | NULL     | NULL           | 0.15                | 2023-01-18 | Post-IPO | United States |
| Carvana             | Phoenix     |          | 2500           | 0.12                | 2022-05-10 | Post-IPO | United States |
| Jul                 | SF Bay Area |          | 400            | 0.3                 | 2022-11-10 | Unknown  | United States |

### 11.3 Finding Matching Records to Populate Missing Industries

Instead of immediately deleting the missing values, the mentor checked whether the same company had another record with a valid industry value.

To fill the missing value safely. To achieve this, a self-join was performed on the same table.

```

167 -- matching rows having missing industry with rows having valid industries for the company and location
168 -- using self join
169
170 • SELECT t1.company, t1.location, t1.industry,
171        t2.company, t2.location, t2.industry
172 FROM layoffs_staging2 t1
173 JOIN layoffs_staging2 t2
174 ON t1.company = t2.company
175 AND t1.location = t2.location
176 WHERE (t1.industry IS NULL OR t1.industry = '')
177 AND t2.industry IS NOT NULL;

```

| company | location    | industry | company | location    | industry       |
|---------|-------------|----------|---------|-------------|----------------|
| Airbnb  | SF Bay Area |          | Airbnb  | SF Bay Area |                |
| Airbnb  | SF Bay Area |          | Airbnb  | SF Bay Area | Travel         |
| Carvana | Phoenix     |          | Carvana | Phoenix     |                |
| Carvana | Phoenix     |          | Carvana | Phoenix     | Transportation |
| Carvana | Phoenix     |          | Carvana | Phoenix     | Transportation |
| Juul    | SF Bay Area |          | Juul    | SF Bay Area |                |
| Juul    | SF Bay Area |          | Juul    | SF Bay Area | Consumer       |

This query matched rows with missing industries to rows containing valid industries for the same company and location.

### 11.4 Converting Blank Values to Null

While updating the missing values the following query was used

```

UPDATE layoffs_staging2 t1
JOIN layoffs_staging2 t2
ON t1.company = t2.company
SET t1.industry = t2.industry
WHERE (t1.industry IS NULL OR t1.industry = '')
AND t2.industry IS NOT NULL;

```

attempted to copy the valid industry value into rows where the industry was missing. However, after running the update query, the missing values were still not updated successfully.

#### Reason for the Issue :

- The problem occurred because some missing industry values were stored as **blank strings ('')** instead of actual NULL values.
- The update condition was checking only for: `WHERE t1.industry IS NULL`

Since blank values are different from NULL in MySQL, those rows were ignored during the update operation.

To fix the problem, all blank industry values were first converted into proper NULL values .

### 11.5 Populating Missing Industry Values

After converting blanks to nulls, the missing industries were updated using values from matching records.

```

185 • UPDATE layoffs_staging2 t1
186 JOIN layoffs_staging2 t2
187 ON t1.company = t2.company
188 SET t1.industry = t2.industry
189 WHERE t1.industry IS NULL
190 AND t2.industry IS NOT NULL;
191
192 • SELECT *
193 FROM layoffs_staging2
194 WHERE company IN ('Airbnb' , 'Carvana' , 'Juil');

```

| company | location    | industry       | total_laid_off | percentage_laid_off | date       | stage          | country       |
|---------|-------------|----------------|----------------|---------------------|------------|----------------|---------------|
| Airbnb  | SF Bay Area | Travel         | 30             | NULL                | 2023-03-03 | Post-IPO       | United States |
| Airbnb  | SF Bay Area | Travel         | 1900           | 0.25                | 2020-05-05 | Private Equity | United States |
| Carvana | Phoenix     | Transportation | 2500           | 0.12                | 2022-05-10 | Post-IPO       | United States |
| Carvana | Phoenix     | Transportation | NULL           | NULL                | 2023-01-13 | Post-IPO       | United States |
| Carvana | Phoenix     | Transportation | 1500           | 0.08                | 2022-11-18 | Post-IPO       | United States |
| Juul    | SF Bay Area | Consumer       | 400            | 0.3                 | 2022-11-10 | Unknown        | United States |
| Juul    | SF Bay Area | Consumer       | 900            | 0.3                 | 2020-05-05 | Unknown        | United States |

Populated industry values after update

This query successfully populated missing industry values for companies such as Airbnb , Carvana and Juul

## 11.6 Handling Remaining Null Values

Some records still contained null values after the update process. For example, certain companies had only one record in the dataset, so there was no matching industry value available to populate the missing data.

It was decided to leave such records unchanged because there was no reliable source within the dataset to infer the missing value.

```

161 -- looking for null and blank in industry column
162 • SELECT *
163 FROM layoffs_staging2
164 WHERE industry IS NULL
165 OR industry = '';
166
167 -- matching rows having missing industry with rows having valid industries for the com
168 -- using self join

```

| company             | location   | industry | total_laid_off | percentage_laid_off | date       | stage    | co  |
|---------------------|------------|----------|----------------|---------------------|------------|----------|-----|
| Bally's Interactive | Providence | NULL     | NULL           | 0.15                | 2023-01-18 | Post-IPO | Uni |

Records with remaining null values

## 11.7 Removing Rows with Completely Missing Layoff Information

Rows where both:

- total\_laid\_off
- percentage\_laid\_off

were null were considered unreliable for analysis. As these two columns could be majorly used for EDA

```
-- looking for rows with completely null layoff info
```

- ```
SELECT *
FROM layoffs_staging2;
```
- ```
SELECT *
FROM layoffs_staging2
WHERE total_laid_off IS NULL
AND percentage_laid_off IS NULL;
```

| company     | location    | industry       | total_laid_off | percentage_laid_off | date       | stage    | country       |
|-------------|-------------|----------------|----------------|---------------------|------------|----------|---------------|
| Inc.        | Toronto     | Transportation | NULL           | NULL                | 2022-12-16 | Post-IPO | Canada        |
| .00 Thieves | Los Angeles | Retail         | NULL           | NULL                | 2023-01-10 | Series C | United States |
| icolade     | Seattle     | Healthcare     | NULL           | NULL                | 2023-03-03 | Post-IPO | United States |
| ida         | Toronto     | Support        | NULL           | NULL                | 2023-02-01 | Series C | Canada        |
| idara       | SF Bay Area | Travel         | NULL           | NULL                | 2020-03-31 | Series C | United States |
| iddi        | Bogota      | Finance        | NULL           | NULL                | 2022-06-14 | Series C | Colombia      |
| irMap       | Los Angeles | Aerospace      | NULL           | NULL                | 2020-04-30 | Unknown  | United States |

Layoff information with null

Since these rows did not provide meaningful layoff information These records could negatively impact analytical accuracy, they were deleted.

```
210 • SELECT *
211 FROM layoffs_staging2
212 WHERE total_laid_off IS NULL
213 AND percentage_laid_off IS NULL;
214
215 -- Delete Useless data that can't really be used
216 • DELETE
217 FROM layoffs_staging2
218 WHERE total_laid_off IS NULL
219 AND percentage_laid_off IS NULL;
220
```

| company | location | industry | total_laid_off | percentage_laid_off | date | stage | country |
|---------|----------|----------|----------------|---------------------|------|-------|---------|
|---------|----------|----------|----------------|---------------------|------|-------|---------|

Deleted all nulls from the both columns

## 12) Removing Helper Column

After cleaning duplicates, the helper column **row\_num** was no longer needed and was removed.

```
221 -- removing helper column no longer needed
222 • ALTER TABLE layoffs_staging2
223 DROP COLUMN row_num;
224
```

| #  | Time     | Action                                                                                 | Message                                                |
|----|----------|----------------------------------------------------------------------------------------|--------------------------------------------------------|
| 50 | 14:30:39 | SELECT * FROM layoffs_staging2 WHERE total_laid_off IS NULL AND percentage_laid_off... | 0 row(s) returned                                      |
| 51 | 14:36:14 | ALTER TABLE layoffs_staging2 DROP COLUMN row_num                                       | 0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0 |

## Final cleaned table structure

```

224
225 • SELECT *
226 FROM layoffs_staging2;
227
228

```

| company         | location        | industry       | total_laid_off | percentage_laid_off | date       | stage    | country       | funds_raised_millions |
|-----------------|-----------------|----------------|----------------|---------------------|------------|----------|---------------|-----------------------|
| Included Health | SF Bay Area     | Healthcare     | NULL           | 0.06                | 2022-07-25 | Series E | United States | 272                   |
| 8Open           | Dublin          | Marketing      | 9              | 0.09                | 2022-11-17 | Series A | Ireland       | 35                    |
| #Paid           | Toronto         | Marketing      | 19             | 0.17                | 2023-01-27 | Series B | Canada        | 21                    |
| 100 Thieves     | Los Angeles     | Consumer       | 12             | NULL                | 2022-07-13 | Series C | United States | 120                   |
| 10X Genomics    | SF Bay Area     | Healthcare     | 100            | 0.08                | 2022-08-04 | Post-IPO | United States | 242                   |
| 1stdibs         | New York City   | Retail         | 70             | 0.17                | 2020-04-02 | Series D | United States | 253                   |
| 2TM             | Sao Paulo       | Crypto         | 90             | 0.12                | 2022-06-01 | Unknown  | Brazil        | 250                   |
| 2TM             | Sao Paulo       | Crypto         | 100            | 0.15                | 2022-09-01 | Unknown  | Brazil        | 250                   |
| 2U              | Washington D.C. | Education      | NULL           | 0.2                 | 2022-07-28 | Post-IPO | United States | 426                   |
| 54gene          | Washington D.C. | Healthcare     | 95             | 0.3                 | 2022-08-29 | Series B | United States | 44                    |
| 5B Solar        | Sydney          | Energy         | NULL           | 0.25                | 2022-06-03 | Series A | Australia     | 12                    |
| 6sense          | SF Bay Area     | Sales          | 150            | 0.1                 | 2022-10-12 | Series E | United States | 426                   |
| 80 Acres Farms  | Cincinnati      | Food           | NULL           | 0.1                 | 2023-01-18 | Unknown  | United States | 275                   |
| 8x8             | SF Bay Area     | Support        | 155            | 0.07                | 2023-01-18 | Post-IPO | United States | 253                   |
| 8x8             | SF Bay Area     | Support        | 200            | 0.09                | 2022-10-04 | Post-IPO | United States | 253                   |
| 98point6        | Seattle         | Healthcare     | NULL           | 0.1                 | 2022-07-21 | Series E | United States | 247                   |
| 99              | Sao Paulo       | Transportation | 75             | 0.02                | 2022-09-20 | Acquired | Brazil        | 244                   |
| Abra            | SF Bay Area     | Crypto         | 12             | 0.05                | 2022-06-30 | Series C | United States | 106                   |
| Absci           | Vancouver       | Healthcare     | 40             | NULL                | 2022-08-09 | Post-IPO | United States | 237                   |
| Acast           | Stockholm       | Media          | 70             | 0.15                | 2022-09-15 | Post-IPO | Sweden        | 126                   |

The final dataset was fully cleaned and ready for:

- Exploratory Data Analysis (EDA)
- Trend analysis
- Visualization
- SQL querying

The cleaning process improved:

- Data consistency
- Accuracy and completeness
- Reliability
- Overall data quality and analytical reliability

## 13) Before vs After Metrics

| <i>Metric</i>                  | <i>Before Cleaning</i> | <i>After Cleaning</i> |
|--------------------------------|------------------------|-----------------------|
| <i>Total Rows</i>              | 2361                   | 1995                  |
| <i>Duplicate Records</i>       | 5                      | 0                     |
| <i>Blank Industries</i>        | 4                      | 1                     |
| <i>Invalid Country Formats</i> | 4                      | 0                     |
| <i>Null Layoff Rows</i>        | 362                    | 0                     |

## 14) Challenges Faced During the Project

- While removing duplicate records, MySQL did not allow deleting rows directly from a CTE because CTEs are not updatable in MySQL. To solve this, an additional staging table with a row\_num column was created to safely identify and remove duplicates.
- Identifying duplicates was initially challenging because some rows appeared similar but were not exact duplicates. The duplicate detection logic had to be improved by partitioning across all important columns.
- Missing values in the industry column were stored both as NULL values and blank strings (''). This caused issues during self-join updates because blank values were not treated as NULL.
- While updating missing industry values using self joins, the update query initially failed because it checked only for NULL values. The issue was resolved by first converting blank values into NULL.
- The date column was imported as TEXT, which prevented proper time-series analysis. The dates first had to be standardized using STR\_TO\_DATE() before changing the datatype to DATE.
- Standardizing text values was challenging because similar industries and country names had inconsistent formats such as:
  - Crypto vs Cryptocurrency
  - United States vs United States.
- Deciding whether to remove rows with missing layoff information required analytical judgment, since deleting data can sometimes remove potentially useful information.
- The project involved many real-world SQL scenarios that were not covered directly in lessons, requiring experimentation, debugging, and problem-solving throughout the cleaning process.